

FTV GraphQL-profiel

Autorisatie per veld van GraphQL-query's, op basis van het NLgov AuthZEN-profiel

Peter-Jan Karens, Lead Architect bij Programma GBO

Draft-01 · FTV-werkgroep · september 2026

Waarom GraphQL met FTV

Eén API per bron

Een GraphQL-API ontsluit een bron eenmalig, voor alle afnemers

Dataminimalisatie

De afnemer vraagt alleen de velden die hij nodig heeft

Toegang via EAM

De aanbieder legt per afnemer en doel vast welke velden zijn toegestaan

Nieuwe afnemers

Aansluiten vraagt een policywijziging, geen nieuwe API

GraphQL: het schema

- Een bron publiceert één schema met types en velden
- Een veld kan naar een ander type verwijzen: adres is van type Adres
- Een veld kan argumenten hebben, zoals bsn en jaren

Schema

```
type Query {  
  persoon(bsn: String!): Persoon  
}  
  
type Persoon {  
  naam: String  
  adres: Adres  
  inkomens(jaren: [Int!]): [Inkomen!]  
}  
  
type Adres {  
  straat: String  
  huisnummer: Int  
  postcode: String  
  plaats: String  
}  
  
type Inkomen {  
  jaar: Int!  
  bedrag: Float  
}
```

Een query en het antwoord

Query

```
query {  
  persoon(bsn: "9999990019") {  
    naam  
    inkomens {  
      jaar  
      bedrag  
    }  
  }  
}
```

- Het antwoord volgt de structuur van de query
- adres is niet opgevraagd en ontbreekt in het antwoord

Antwoord van de bron

```
{  
  "data": {  
    "persoon": {  
      "naam": "J. Jansen",  
      "inkomens": [  
        {  
          "jaar": 2024,  
          "bedrag": 41250.0  
        }  
      ]  
    }  
  }  
}
```

GraphQL-over-HTTP request

HTTP-verzoek

```
POST /graphql HTTP/1.1
Content-Type: application/json

{
  "query": "query($bsn: String!) { persoon(bsn: $bsn) { naam inkomens { jaar bedrag } } }",
  "variables": {
    "bsn": "999990019"
  }
}
```

- Elk verzoek is een POST naar hetzelfde endpoint
- De gevraagde velden staan als tekst in het veld `query`
- Waarden zoals het BSN staan apart in `variables`
- Zonder parser en schema kan een PEP of policy het verzoek niet beoordelen

Opties voor autorisatie bij GraphQL

Vóór het endpoint

Een standaard-PEP vraagt de PDP om één besluit per verzoek.

In de resolvers

De bron doet zelf AuthZEN-calls per object of veld, op meerdere niveaus van de query.

Stitching of federatie

Een gateway voegt meerdere endpoints samen tot één schema. Autorisatie in de gateway of per endpoint.

- Dit voorstel: AuthZEN generiek vóór een GraphQL-endpoint (waarvan evt. de implementatie niet in eigen beheer is)
- De andere opties zijn ook geldig, maar vallen buiten dit voorstel
- Combineren kan: dit profiel vóór een gateway, of extra controles in de bron

Uitgangspunten

- De mapper verwerkt elke geldige GraphQL-query en werkt fail-closed
- Alleen de policies bepalen de toegang. De PAP laadt ze in de PDP (EAM / PxP)
- Eén query leidt tot één decision: alles of niets
- Een veld zonder regel is niet toegankelijk (deny by default)
 - een GraphQL-API heeft geen versies en kan op elk moment velden toevoegen
- Bron en policies hebben elk een eigen deploymentcyclus
- De bron voert precies het verzoek uit dat de PDP heeft beoordeeld
 - de bron accepteert daarom alleen POST met JSON op één pad

Afgewezen alternatieven

Alternatief	Reden
Mapping in de PEP met COAZ	COAZ ondersteunt alleen eenvoudige mappings. De velden van een query zijn pas bij het verzoek bekend.
Custom PEP-extensie voor GraphQL	De PEP blijft bij voorkeur een standaardcomponent. Het schema hoort bij de policies in de bundel, zodat beide tijdens de uitvoering vastliggen.
GraphQL parsen in de policies	Alleen OPA kan GraphQL parsen, en de parseboom bevat geen types. Elke bundel heeft dan een omvangrijke, generieke engine.rego nodig.

Voorstel: een mapper bij de PDP



- De mapper draait in of vóór de PDP, als plugin of als middleware
- De policies lezen alleen de veldlijst, geen GraphQL
- De mapper kent het schema, niet de toegangsregels

Wat de mapper toevoegt

Query

```
query {  
  persoon(bsn: "9999990019") {  
    naam  
    adres {  
      plaats  
    }  
  }  
}
```

- De mapper maakt één record per veld in de query
- Het type komt uit het schema, niet uit de query
- De lijst staat in `resource.properties.graphql`

Toegevoegd door de mapper

```
{  
  "fields": [  
    { "path": ["persoon"], "parentType": "Query", "field": "persoon", "leaf": false,  
      "args": { "bsn": { "value": "9999990019", "origin": "literal" } } },  
    { "path": ["persoon", "naam"], "parentType": "Persoon", "field": "naam", "leaf": true },  
    { "path": ["persoon", "adres"], "parentType": "Persoon", "field": "adres", "leaf": false },  
    { "path": ["persoon", "adres", "plaats"], "parentType": "Adres", "field": "plaats", "leaf": true }  
  ]  
}
```

Regels bij het schema

Schema

```
type Query {  
  persoon(bsn: String!): Persoon  
}  
type Persoon {  
  naam: String  
  adres: Adres  
  inkomens(jaren: [Int!]): [Inkomen!]  
}  
type Adres {  
  straat: String  
  huisnummer: Int  
  postcode: String  
  plaats: String  
}  
type Inkomen {  
  jaar: Int!  
  bedrag: Float  
}
```

Regels (pseudocode)

```
regel persoon-op-bsn  
  velden: Query.persoon  
  conditie: toestemming voor bsn  
  
regel naam-en-adres  
  velden: Persoon.naam  
         Persoon.adres # objectveld: erft nooit  
  types: Adres # de velden van Adres erven  
  conditie: afnemer geautoriseerd
```

- Elke policy-engine is mogelijk: het regelformaat is vrij
- De velden binnen Adres erven de regel via het type
- Objectvelden, velden op Query of met argumenten erven nooit
- inkomens staat in geen regel

Toegestaan en geweigerd

Toegestaan

```
query {  
  persoon(bsn: "999990019") { # persoon-op-bsn  
    naam # naam-en-adres  
    adres { # naam-en-adres  
      straat # naam-en-adres, via Adres  
      plaats # naam-en-adres, via Adres  
    }  
  }  
}
```

Geweigerd

```
query {  
  persoon(bsn: "999990019") { # persoon-op-bsn  
    naam # naam-en-adres  
    inkomens { # geen regel  
      jaar # geen regel  
    }  
  }  
}
```

- De PDP beoordeelt elk veld apart, met de regels voor zijn type en veldnaam
- Links heeft elk veld een regel die **allow** geeft: de query is toegestaan
- Rechts hebben inkomens en jaar geen regel: **deny** voor de hele query

Uitkomst en foutmelding

- Alle velden toegestaan: **allow**
- Eén veld niet toegestaan: **deny**
voor de hele query
- Niet-ondersteunde vorm of
verwerkingsfout: **deny**
- De afnemer ziet het eerste veld
dat de weigering veroorzaakt
- Voorbeeld: de geweigerde query
van de vorige dia

Antwoord aan de afnemer

```
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "errors": [
    {
      "message": "Field not permitted: persoon.inkomens",
      "extensions": {
        "code": "FIELD_NOT_PERMITTED",
        "field_id": "persoon.inkomens"
      }
    }
  ]
}
```

Argumenten controleren

Extra regels voor inkomens (pseudocode)

```
regel inkomens-tot-en-met-2024  
  velden: Persoon.inkomens  
  conditie: jaren staat in de query  
            en elk jaar <= 2024
```

```
regel inkomen-velden  
  velden: Inkomen.jaar, Inkomen.bedrag  
  conditie: afnemer geautoriseerd
```

Selectie	Uitkomst
<code>inkomens(jaren: [2024])</code>	allow
<code>inkomens(jaren: [2025])</code>	deny
<code>inkomens</code>	deny: jaren ontbreekt

- De conditie leest de waarde uit de argumenten van het veld zelf
- Ontbreekt de waarde of komt die uit een schema-default: **deny**
 - geen vertrouwen in schema-defaults: door schema drift kan de bron een andere default hebben

Voorbeeld in Rego: een regel

rules/persoon-op-bsn.rego

```
# Een persoon mag alleen met toestemming
# voor het gevraagde BSN.
package rules["persoon-op-bsn"]

import input.field
import input.subject

covers_fields := {"Query.persoon"}

allow if {
  bsn := field.args.bsn.value
  bsn in data.toestemming[subject.id]
}
```

Gegevens in de PDP

```
"toestemming": {
  "0000000010000000000001": ["999990019"]
}
```

Verzoek aan de PDP, na de mapper (ingekort)

```
{
  "subject": { "id": "0000000010000000000001" },
  "resource": { "properties": { "graphql": {
    "fields": [
      { "path": ["persoon"],
        "parentType": "Query", "field": "persoon",
        "args": { "bsn": { "value": "999990019",
                          "origin": "literal" } } },
      ...
    ]
  } } }
}
```

- allow geldt voor elk veld met een sleutel uit covers_fields
- field is dat veld uit de lijst van de mapper
- subject is de afnemer die de PEP meestuurt

Voorbeeld in Rego: meer regels

rules/naam-en-adres.rego

```
# Naam en adres mogen als de afnemer
# een autorisatie voor deze regel heeft.
package rules["naam-en-adres"]

import data.autorisaties
import input.subject

# adres is een objectveld: erft nooit
covers_fields := {"Persoon.naam", "Persoon.adres"}

# velden van Adres zonder subvelden en
# argumenten erven deze regel
covers_types := {"Adres"}

allow if {
  subject.id in autorisaties["naam-en-adres"]
}
```

rules/inkomens-tot-en-met-2024.rego

```
# Inkomens alleen voor jaren die in de
# query staan, tot en met 2024.
package rules["inkomens-tot-en-met-2024"]

import input.field

covers_fields := {"Persoon.inkomens"}

allow if {
  jaren := field.args.jaren
  jaren.origin != "schema-default"
  every jaar in jaren.value { jaar <= 2024 }
}
```

- De velden van Adres erven via covers_types, het objectveld adres niet
- Ontbreekt jaren of komt het uit een schema-default, dan is allow niet waar: deny

Het vaste deel: regels koppelen

ftv/graphql.rego: velden en sleutels

```
package ftv.graphql

gql := input.resource.properties.graphql

# data-velden: zonder __typename en introspectie
data_fields contains i if {
    some i, f in gql.fields
    f.field != "__typename"
    not startswith(f.parentType, "__")
}

key(f) := concat(".", [f.parentType, f.field])

# alle sleutels die een regel noemt
field_keys contains k if {
    some k in data.rules[_].covers_fields
}

# de standaardnamen van de root-types
root_types := {"Query", "Mutation", "Subscription"}
```

ftv/graphql.rego: regels koppelen

```
# een regel met de sleutel van het veld geeft allow
allowed(f) if {
    some id
    key(f) in data.rules[id].covers_fields
    data.rules[id].allow with input.field as f
}

# of: een veld zonder subvelden, argumenten en
# eigen regel erft de regels van zijn type
allowed(f) if {
    f.leaf
    not f.args
    not key(f) in field_keys
    not f.parentType in root_types
    some id
    f.parentType in data.rules[id].covers_types
    data.rules[id].allow with input.field as f
}
```

Het vaste deel: besluit en antwoord

ftv/graphql.rego: besluit

```
denied contains i if {
  some i in data_fields
  not allowed(gql.fields[i])
}

default decision := false

decision if {
  gql.profile == "ftv-graphql/0.1"
  gql.unverifiable == null
  count(data_fields) > 0
  count(denied) == 0
}
```

ftv/graphql.rego: antwoord aan de PEP

```
# antwoord aan de PEP, via de AuthZEN-adapter
response := {"decision": true} if decision

response := {
  "decision": false,
  "context": {"graphql": {"client": client}},
} if not decision
```

- Dit deel is voor elke bron gelijk
- Een nieuwe regel is een nieuw bestand in rules/
- Verder nodig: de mapper en een AuthZEN-adapter voor OPA

Randgevallen in de query

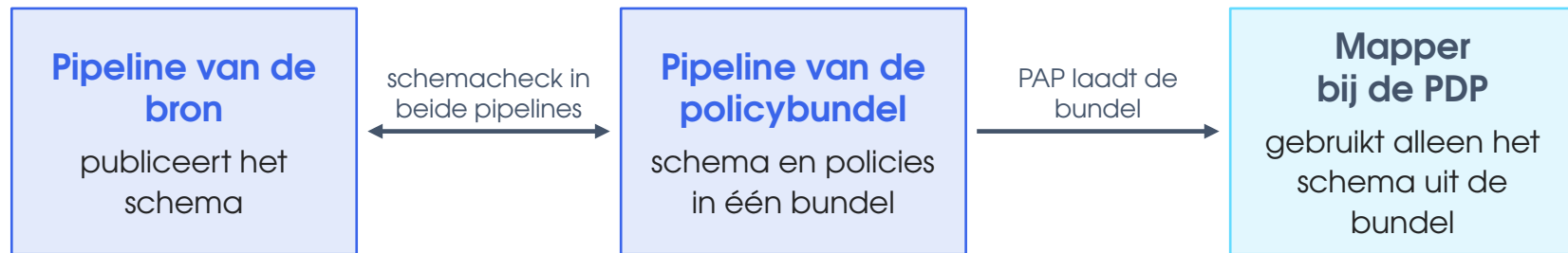
Geval	Voorbeeld	Aanpak
Aliassen	a: persoon(...) b: persoon(...)	Elke selectie is een eigen record met eigen argumenten. De binding negeert de alias.
Fragmenten	...PersoonVelden	De mapper vouwt fragmenten uit. Twee keer gebruikt is twee keer beoordeeld.
Interfaces en unions	... on LoonInkomen { bedrag }	De sleutel gebruikt het concrete type. Zonder on geldt de sleutel op de interface.
@skip en @include	naam @skip(if: \$x)	Het veld telt altijd mee. Andere directives, zoals @defer: deny .
Variabelen en defaults	(\$jaar: Int = 2024)	De mapper noteert de herkomst. Schema-default niet vertrouwd (schema drift): deny .
Introspectie	__typename, __schema	__typename valt weg. __schema en __type hebben een eigen regel nodig.

Randgevallen in het verzoek

Geval	Uitkomst
GET, query string, ander pad of media type	<code>deny: UNSUPPORTED_TRANSPORT</code>
Batch, persisted query, dubbele JSON-sleutel	<code>deny: INVALID_BODY</code>
Meer operaties zonder operationName	<code>deny: OPERATION_AMBIGUOUS</code>
Mutation of subscription	<code>deny: OPERATION_NOT_SUPPORTED</code>
Parsefout, onbekend veld of fragment	<code>deny: PARSE_ERROR, INVALID_QUERY</code>
Variabele met een verkeerd type	<code>deny: VARIABLE_ERROR</code>
Te diep, te veel velden of aliases	<code>deny: LIMIT_EXCEEDED</code>
Schema ontbreekt of digest wijkt af	<code>deny: CONFIG_ERROR</code>

Schema in de policybundel

- De mapper gebruikt het schema uit de policybundel, niet het live schema
- Een nieuw veld in de bron leidt tot deny, totdat de bundel het bevat
- Een schemacheck vangt wijzigingen af die niet in de query zichtbaar zijn
 - bijvoorbeeld een nieuw argument of een gewijzigde default



Bedankt! Vragen?